

Full Length Article

Improving human motion generation based on a head-mounted display and its controllers via noise-augmented motion data and the recurrent inference model

Zihao Guo ^a, Jingbo Zhao ^{a,b,*}

^a College of Information and Electrical Engineering, China Agricultural University, Beijing, China

^b National Innovation Center for Digital Agricultural Products Circulation, China Agricultural University, Beijing, China

ARTICLE INFO

Keywords:

3-D human motion generation
Conditional variational autoencoder
Head-mounted display

ABSTRACT

Previous full-body human motion generation methods based on a head-mounted display (HMD) and two hand-held controllers rely on the assumption that the connections between the devices and body parts are rigid. Such approaches are not robust to motion noises, which are frequently introduced during virtual reality (VR) experience as these devices are often loosely attached to body parts. To address this problem, we propose a new generative strategy that synthesizes noise-augmented motion data to enhance the performance of human motion generation models and compared it against other three training strategies, including the direct, fine-tuning, and additive strategies. Furthermore, we present the Recurrent Inference Model (RIM), an approach that further improves the performance of full-body human motion generation. Our RIM model trained on noise-augmented motion data demonstrates consistent improvements over other existing state-of-the-art (SOTA) human motion generation methods during offline and real-time evaluation. The code of this research is available at: <https://github.com/vrlab561/RIM-release>.

1. Introduction

Head-mounted displays (HMDs) paired with their two hand-held controllers are standard configurations for virtual reality (VR) systems. Full-body human motion generation based on an HMD and its controllers (also referred to as VR devices in the present paper) plays an important role in enabling immersive experiences across VR games and social interactions, etc. Previous full-body human motion generation methods (Aliakbarian et al., 2023; Barquero et al., 2025; Dai et al., 2024; Dittadi et al., 2021; Du et al., 2023; Jiang et al., 2022a; Zheng et al., 2023) have not considered the non-rigid connections between devices and body parts. These non-rigid connections are caused by the loose coupling between the devices and their associated body parts and can be expressed as non-rigid transformations between the coordinate systems of the devices and body parts in terms of translations and rotations (see Fig. 1).

Large-scale motion capture (MoCap) datasets, such as AMASS (Mahmood et al., 2019), that are often used for training full-body motion generation models are inferred from optical tracking data and do not account for such non-rigid transformations. These inferred motion data can be referred to as synthetic data. Consequently, models trained on

such MoCap datasets may exhibit degraded performance when deployed on HMDs in real-world application scenarios as these methods are not robust to the motion noises introduced by the non-rigid transformations. Existing datasets, such as the FreeDancing dataset (Dai et al., 2024), captured on a setup of an HMD and its controllers are small-scale. They contain limited data that cover only a small range of human motions. Full-body motion generation models trained on such datasets may not generate a wide range of human motions.

To address this problem, we propose a new generative strategy that synthesizes noise-augmented motion data by learning the non-rigid positional and rotational transformations between the devices and their associated body parts based on a conditional variational autoencoder (CVAE) (Sohn et al., 2015). The trained CVAE model then generates non-rigid transformations and applies these transformations to the synthetic MoCap data in the AMASS dataset (Mahmood et al., 2019) during training time. This strategy effectively augments existing synthetic data and produces training motion data that resemble the motion data captured on an HMD and its controllers. To validate the effectiveness of the proposed generative approach, we compared it against three other training strategies, including the direct, fine-tuning, and additive strategies. Furthermore, we propose an improved full-body motion generation model

* Corresponding author.

E-mail addresses: guozihao010@gmail.com (Z. Guo), zhao.jingbo@cau.edu.cn (J. Zhao).

<https://doi.org/10.1016/j.neunet.2026.108918>

Received 23 October 2025; Received in revised form 7 February 2026; Accepted 29 March 2026

Available online 31 March 2026

0893-6080/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

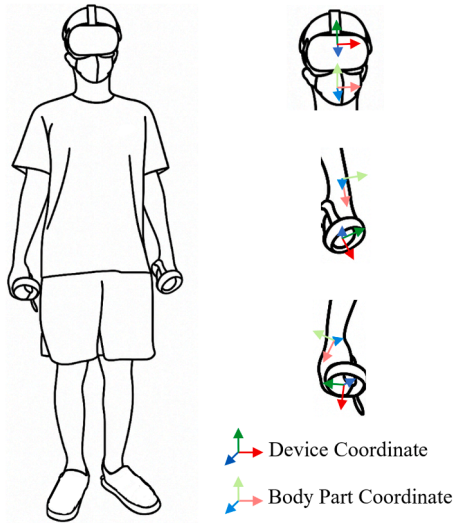


Fig. 1. Schematic of the coordinate systems of the VR devices and their associated body parts. Since VR devices are often loosely attached to body parts, the connections are non-rigid. These connections can be expressed as non-rigid positional and rotational transformations between the devices and body parts.

called the Recurrent Inference Model (RIM). Inspired by HMD-NeMo (Aliakbarian et al., 2023) and HMD-Poser (Dai et al., 2024), our improved model employs a spatial-temporal feature extraction backbone based on long short-term memory (LSTM) networks and transformers. In addition, it incorporates the Model Initialization module that uses both initial local poses and global poses to initialize the states of LSTMs in the backbone. Compared to the multi-layer perceptron (MLP) regressor of HMD-Poser (Dai et al., 2024) that processes individual frame output from the transformer encoder, the Temporal Regression module accounts for the temporal dependencies of the features produced by the backbone and improves the accuracy and smoothness of generated human motion. During the model inference stage, the states of LSTMs in the backbone and Temporal Regression are preserved after processing each motion subsequence and are used for self-initialization before the inference of full-body motions using the next subsequence, thereby establishing temporal dependencies of input subsequences.

The contributions of our research are:

- We propose a new generative strategy to synthesize noise-augmented motion data based on a CVAE model. This strategy effectively enhances the performance of human motion generation models.
- We present three other training strategies, including the direct, fine-tuning, and additive strategies and compared the generative strategy against these three strategies to demonstrate the superiority of the generative strategy.
- We propose an improved full-body motion generation model called RIM. RIM incorporates three key design components, including Model Initialization, Temporal Regression and Recurrent Inference. RIM achieves SOTA results during offline and real-time evaluations.

2. Related work

2.1. Human motion generation based on an HMD and its two hand-held controllers

Generating full-body human motion using only an HMD and its controllers without additional trackers offers convenience and is practical for VR experience. Existing methods fall into three categories. The first category of methods (Aliakbarian et al., 2022; Dittadi et al., 2021; Du et al., 2023; Starke et al., 2024) adopted generative models to generate motion. These include variational autoencoder (VAE) (Dittadi et al.,

2021), normalizing flows (Aliakbarian et al., 2022), diffusion (Du et al., 2023), and categorical codebook matching (Starke et al., 2024). The second category of methods (Lee et al., 2023; Winkler et al., 2022) utilized reinforcement learning to simulate human motions. QuestSim (Winkler et al., 2022) learned the control policy of the avatar from the motion tracking data of an HMD and its controllers while QuestEnvSim (Lee et al., 2023) extended this framework by incorporating interactions between the avatar and the environment. Other methods (Aliakbarian et al., 2023; Barquero et al., 2025; Dai et al., 2024; Jiang et al., 2022a; Zheng et al., 2023) were based on deep learning models. AvatarPoser (Jiang et al., 2022a), AvatarJLM (Zheng et al., 2023) and rolling prediction models (RPM) (Barquero et al., 2025) employed pure transformer-based backbone networks, whereas HMD-NeMo (Aliakbarian et al., 2023) and HMD-Poser (Dai et al., 2024) used hybrid architectures that integrated gated recurrent units (GRUs) or LSTMs with transformer modules. However, existing methods did not consider the non-rigid connections between VR devices and the corresponding body parts in real-world application scenarios. Their performance may degrade during real-time operation.

2.2. Data augmentation for human motion generation based on VR devices

Previous work (Dai et al., 2024) has pointed out that the non-rigid connections between users' hands and the hand-held controllers may lead to degraded model performance in real-world scenarios. However, the solutions to this problem remain unexplored. Motion data augmentation was only investigated in some other specific scenarios. For example, HMD-NeMo (Aliakbarian et al., 2023) and RPM (Barquero et al., 2025) focused on data augmentation for hand visibility problems during hand tracking. HMD-NeMo (Aliakbarian et al., 2023) simulated the field of view (FoV) frustums of HMD cameras to model the hand visibility status while RPM (Barquero et al., 2025) simulated random tracking signal losses lasting from 0.5 s to 2 s. Motion data augmentation has been studied in motion generation based on inertial measurement units (IMUs) (Li et al., 2024; Zuo et al., 2024). PoseAugment (Li et al., 2024) introduced an architecture that integrated VAE-based pose generation with physical optimization. This method can generate realistic and diverse pose sequences with input motions. Loose Inertial Poser (LIP) (Zuo et al., 2024) specifically targeted scenarios where IMUs were attached to loose jackets and utilized TailorNet (Patel et al., 2020) to simulate garment deformation over SMPL models (Loper et al., 2023). The Secondary Motion AutoEncoder (SeMo-AE) framework was introduced to synthesize the secondary motion between skins and loose jackets. To the best of our knowledge, our generative data augmentation strategy represents the first approach that learns the non-rigid positional and rotational transformations between VR devices and their corresponding body parts and synthesizes noise-augmented motion data during training time.

3. Method

3.1. Problem formulation

Task. Our goal is to generate full-body human motion sequences, including full-body poses and body shapes (Loper et al., 2023), using the tracking data from an HMD and two hand-held controllers, which provide positions (represented as a 3-D vector) and rotations (represented as a 6-D orthogonal transformation (Zhou et al., 2019)) in the global coordinate. The tracked motion data sequence with a length of L from an HMD and two controllers can be represented as $\{\mathbf{p}_d^i, \mathbf{R}_d^i\}_{i=1}^3 \in \mathbb{R}^{L \times 27}$. A single output frame includes the local poses (represented as 6-D orthogonal transformations) of the first 22 joints of the SMPL (Loper et al., 2023) model and the body shape that includes 16 shape parameters. The output pose sequence and the output shape sequence are denoted as $\Theta \in \mathbb{R}^{L \times 132}$ and $\beta \in \mathbb{R}^{L \times 16}$, respectively.

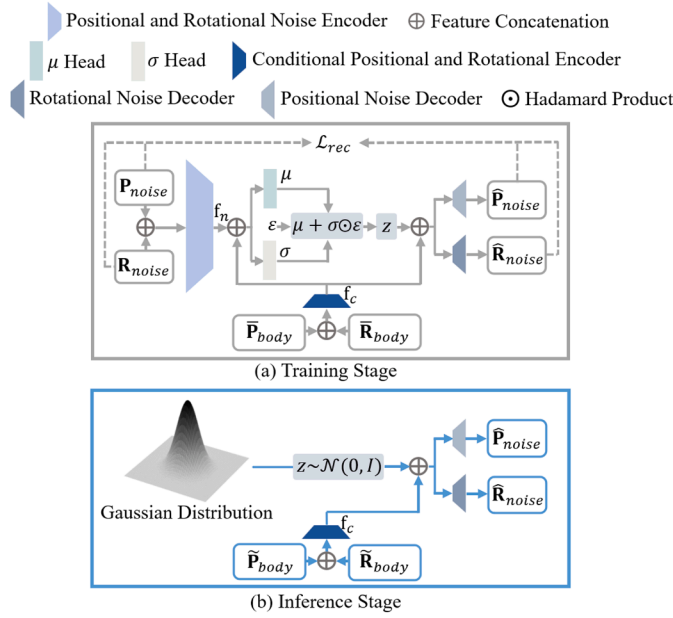


Fig. 2. The architecture of the proposed conditional noise generation model based on the CVAE model. (a) The training process of the model. (b) The inference process of the model. $\{\mathbf{P}_{body}, \mathbf{R}_{body}\}$ and $\{\hat{\mathbf{P}}_{body}, \hat{\mathbf{R}}_{body}\}$ represent the data sequence from an HMD and its controllers during training stage and inference stage, respectively. $\{\mathbf{P}_{noise}, \mathbf{R}_{noise}\}$ and $\{\hat{\mathbf{P}}_{noise}, \hat{\mathbf{R}}_{noise}\}$ represent the ground truth and output positional and rotational noises, respectively.

Non-rigid transformations and motion noise. VR devices are often loosely attached to body parts during VR experience. This results in the non-rigid connections between VR devices and the associated body parts and generates noise to human motion generation. Given the coordinate systems of VR devices and the associated body parts, as shown in Fig. 1, these non-rigid connections can be modeled as non-rigid transformations between VR devices and the corresponding body parts. For VR devices, including an HMD and its controllers, the noise can be modeled as a 3-D positional transformation and a 3-D rotational transformation. Positional and rotational noises can be computed from the Mo-Cap data of a VR device and the ground truth motion data captured by optical trackers or IMUs. Specifically, given the single frame position $\mathbf{p}_d \in \mathbb{R}^3$ and rotation $\mathbf{R}_d \in \mathbb{R}^{3 \times 3}$ of a single VR device, and the position $\mathbf{p}_b \in \mathbb{R}^3$ and rotation $\mathbf{R}_b \in \mathbb{R}^{3 \times 3}$ of the corresponding body part in the ground truth data, the positional noise $\mathbf{p}_n \in \mathbb{R}^3$ and the rotational noise $\mathbf{R}_n \in \mathbb{R}^{3 \times 3}$ can be defined as:

$$\mathbf{p}_n = \mathbf{p}_d - \mathbf{p}_b, \mathbf{R}_n = \mathbf{R}_b^{-1} \mathbf{R}_d. \quad (1)$$

For all three VR devices, including an HMD and two hand-held controllers, the positional noise sequence can be denoted as $\mathbf{P}_{noise} = \{\mathbf{p}_n^i\}_{i=1}^3 \in \mathbb{R}^{L \times 9}$ and the rotational noise sequence can be represented as 6-D orthogonal transformations: $\mathbf{R}_{noise} = \{\mathbf{R}_n^i\}_{i=1}^3 \in \mathbb{R}^{L \times 18}$.

Given the position noise $\mathbf{p}_n$ and rotation noise $\mathbf{R}_n$ of a VR device, and the position $\mathbf{p}_b$ and rotation $\mathbf{R}_b$ of the corresponding body part captured by optical trackers or IMUs, the augmented position $\hat{\mathbf{p}}_b$ and rotation $\hat{\mathbf{R}}_b$ can be calculated using the following equations:

$$\hat{\mathbf{p}}_b = \mathbf{p}_n + \mathbf{p}_b, \hat{\mathbf{R}}_b = \mathbf{R}_b \mathbf{R}_n. \quad (2)$$

Motion data produced by this data augmentation method more closely resemble the data captured by an HMD and its controllers during VR usage. This equation is used in our motion data augmentation and training strategies, including the additive strategy (see Section 4.4) and the generative strategy (see Section 3.2). The additive strategy samples noise directly from the real device dataset while the generative strategy generates noise from a trained noise generation model.

3.2. Noise generation model and motion data augmentation

The noise generation model shall use the positions and rotations of the head and hands as the condition and augments the input data during the training phase of the motion generation models. Given the 3-D representation of the body parts' positions $\{\mathbf{p}_b^i\}_{i=1}^3 \in \mathbb{R}^{n \times 9}$ (in our implementation, $n = 40$) and the 6-D orthogonal transformation of the body parts' rotations $\{\mathbf{R}_b^i\}_{i=1}^3 \in \mathbb{R}^{n \times 18}$, the condition can be represented as $\{\mathbf{p}_b^i, \mathbf{R}_b^i\}_{i=1}^3 \in \mathbb{R}^{n \times 27}$. We use $\mathbf{P}_{body} = \{\mathbf{p}_b^i\}_{i=1}^3$ and $\mathbf{R}_{body} = \{\mathbf{R}_b^i\}_{i=1}^3$ to denote the positions and poses of the body parts.

We propose a conditional noise generation model based on the CVAE model for the generative training strategy. The architecture is illustrated in Fig. 2. The noise generation model consists of the Positional and Rotational Noise Encoder E_n , the Conditional Positional and Rotational Encoder E_c , the μ Head H_μ , the σ Head H_σ , the Positional Noise Decoder D_p , and the Rotational Noise Decoder D_r . E_n , E_c , H_μ , H_σ , D_p , and D_r are all implemented as MLPs. During the training stage, the ground-truth positional and rotational noise sequences are first concatenated and flattened into a vector of size $27n$, and then fed into E_n . E_n maps the noise vector into a D -dimensional hidden feature $\mathbf{f}_n$. The positions and rotations of the head and hands, synthesized from the SMPL motion sequence (Loper et al., 2023), are also concatenated and flattened into a vector of size $27n$. E_c maps the condition vector into a F -dimensional hidden feature $\mathbf{f}_c$. The $\mathbf{f}_n$ and $\mathbf{f}_c$ are concatenated together and serves as input into H_μ and H_σ . H_μ and H_σ estimate the mean μ and standard deviation σ of a Gaussian distribution, respectively. The latent vector $\mathbf{z}$ is then derived from μ and σ through the reparametrization trick:

$$\mathbf{z} = \mu + \sigma \odot \epsilon, \text{ where } \epsilon \sim \mathcal{N}(0, I). \quad (3)$$

The latent vector $\mathbf{z}$ and condition feature $\mathbf{f}_c$ are concatenated and passed through D_p and D_r to output the reconstructed positional noise $\hat{\mathbf{P}}_{noise}$ and rotational noise $\hat{\mathbf{R}}_{noise}$.

Given the estimated Gaussian distribution $p_1 \sim \mathcal{N}(\mu, \sigma)$ and the standard Gaussian prior distribution $p_2 \sim \mathcal{N}(0, I)$, the loss function for training the CVAE model is defined as:

$$\mathcal{L}_{cvae} = \|\hat{\mathbf{P}}_{noise} - \mathbf{P}_{noise}\|_1 + \|\hat{\mathbf{R}}_{noise} - \mathbf{R}_{noise}\|_1 + \alpha_{kl} KL(p_1 || p_2), \quad (4)$$

where KL denotes the Kullback-Leibler divergence (Kullback & Leibler, 1951) and α_{kl} is a hyper-parameter.

The trained noise generation model is employed for data augmentation during the training of downstream motion generation models. During the inference phase, latent vector $\mathbf{z}$ is sampled from the standard Gaussian prior distribution. $\hat{\mathbf{P}}_{body}$ and $\hat{\mathbf{R}}_{body}$ are concatenated and flattened, then fed into E_c , producing the condition feature $\mathbf{f}_c$. The latent vector $\mathbf{z}$ and $\mathbf{f}_c$ are concatenated together to serve as the input for D_p and D_r .

During the training of downstream motion generation models, each batch of sparse observations $\{\hat{\mathbf{P}}_{body}, \hat{\mathbf{R}}_{body}\}$ are augmented with positional and rotational noise from the noise generation model. Specifically, $\{\hat{\mathbf{P}}_{body}, \hat{\mathbf{R}}_{body}\}$ are fed into the noise generation model NG , yielding the positional noise and the rotational noise. Subsequently, the two types of noises are applied to the positions and rotations of each body part individually, transforming the original position and rotation into their augmented counterparts $\hat{\mathbf{P}}_{body}$ and $\hat{\mathbf{R}}_{body}$. The augmented $\hat{\mathbf{P}}_{body}$ and $\hat{\mathbf{R}}_{body}$ then replace the original $\hat{\mathbf{P}}_{body}$ and $\hat{\mathbf{R}}_{body}$ in the input processing stage of the downstream model, where the input processing procedure $Prepare()$ is applied to generate the final input S for the downstream motion generation model.

The training process of the downstream motion generation model MG_θ remains consistent with the original process of the comparison methods, including AvatarPoser (Jiang et al., 2022a), AGRoL (Du et al., 2023), HMD-Poser (Dai et al., 2024), RPM (Barquero et al., 2025). The training of the downstream model involves optimizing the loss function $\mathcal{L}$ with the ground truth motion data M to update the parameters of MG_θ . The entire motion data augmentation process is described in Algorithm 1.

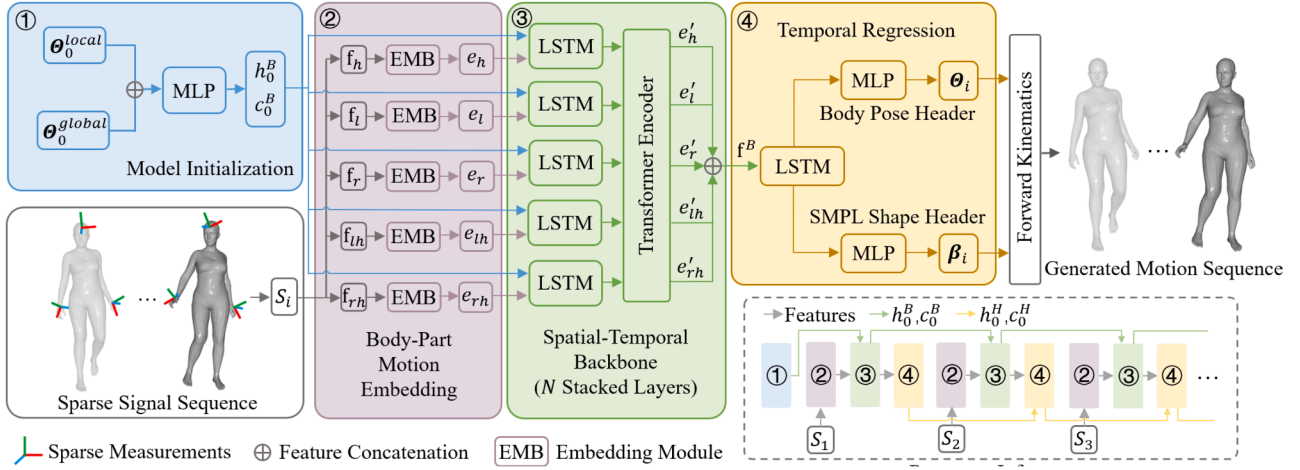


Fig. 3. The RIM architecture, which consists of Model Initialization, Body-Part Motion Embedding, Spatial-Temporal Backbone, and Temporal Regression. Θ_0^{local} and Θ_0^{global} represent the first frame of full-body local pose and global pose. h_0^B and c_0^B represent the initial hidden state and cell state of the i th LSTM in the Backbone. $\{e_h, e_l, e_r, e_{lh}, e_{rh}\}$ represent the body-part features from Embedding Modules. $\{e'_h, e'_l, e'_r, e'_{lh}, e'_{rh}\}$ represent the body-part features from the Backbone. S_i represents the i th input signal subsequence, while Θ_i and β_i represent the corresponding output body pose and SMPL shape subsequences, respectively.

Algorithm 1 Training with the generative strategy.

Require: positions of body parts $\hat{\mathbf{P}}_{body} = \{\hat{\mathbf{P}}_b^i\}_{i=1}^3$; rotations of body parts $\hat{\mathbf{R}}_{body} = \{\hat{\mathbf{R}}_b^i\}_{i=1}^3$; pre-trained noise generation model NG ; input processing procedure $Prepare()$; downstream motion generation model MG_θ ; loss function $\mathcal{L}$; ground truth motion M ;

- 1: # = = = = Generation of noise-augmented motion data = = = =
- 2: **with** torch.no_grad():
- 3: $\hat{\mathbf{P}}_{noise}, \hat{\mathbf{R}}_{noise} = NG(\hat{\mathbf{P}}_{body}, \hat{\mathbf{R}}_{body})$,
- 4: $\{\hat{\mathbf{P}}_b^i\}_{i=1}^3 = \hat{\mathbf{P}}_{noise}$, $\{\hat{\mathbf{R}}_b^i\}_{i=1}^3 = \hat{\mathbf{R}}_{noise}$
- 5: **for each** $i \in [1, 3]$ **do**:
- 6: $\hat{\mathbf{P}}_b^i = \hat{\mathbf{P}}_b^i + \hat{\mathbf{P}}_b^i$, $\hat{\mathbf{R}}_b^i = \hat{\mathbf{R}}_b^i + \hat{\mathbf{R}}_b^i$
- 7: **end for**
- 8: $\hat{\mathbf{P}}_{body} = \{\hat{\mathbf{P}}_b^i\}_{i=1}^3$, $\hat{\mathbf{R}}_{body} = \{\hat{\mathbf{R}}_b^i\}_{i=1}^3$
- 9: $S = Prepare(\hat{\mathbf{P}}_{body}, \hat{\mathbf{R}}_{body})$
- 10: # = = = = = Training downstream model = = = = =
- 11: **with** torch.grad():
- 12: $\hat{M} = MG_\theta(S)$
- 13: Calculate $\mathcal{L}(\hat{M}, M)$
- 14: Update parameters θ of MG_θ

3.3. Recurrent inference model

Overview. We propose the Recurrent Inference Model (RIM) to generate full-body motion sequence. RIM takes motion tracking data sequence as input, including the positions and rotations of tracked body parts, and outputs the corresponding full-body human pose sequence Θ and human shape parameter sequence β . The architecture of the proposed RIM is shown in Fig. 3. It consists of the following four components: (1) Model Initialization, which employs a three-layer MLP architecture to generate the initial hidden states h_0^B and initial cell states c_0^B for the i th LSTM in the Spatial-Temporal Backbone; (2) Body-Part Motion Embedding, which utilizes a three-layer MLP architecture to embed the sparse representations of each body part; (3) Spatial-Temporal Backbone, in which one-directional LSTM is employed to perform temporal modeling on each body-part motion embedding sequence. Then, each output sequence is treated as a single token, and a transformer encoder is utilized to capture the spatial dependencies among all body-part motion embedding sequences; (4) Temporal Regression, including the Body Pose Header and the SMPL Shape Header, which takes the feature se-

quence from Spatial-Temporal Backbone as input and output the local pose sequence and the SMPL shape sequence, respectively.

Model initialization. Learning-based motion initialization methods have been widely adopted in the field of motion generation (Barquero et al., 2025; Jiang et al., 2022b; Van Wouwe et al., 2024; Yi et al., 2022; Zhang et al., 2024). Motivated by previous methods (Yi et al., 2022; Zhang et al., 2024), we adapt the LSTM initialization approach using the initial full-body pose. In our approach, both local and global poses are jointly utilized to generate the initial states for the LSTMs in the Spatial-Temporal Backbone. Specifically, given an initial local pose frame $\Theta_0^{local} \in \mathbb{R}^{132}$ (typically in the N-pose or the T-pose), the corresponding initial global pose frame $\Theta_0^{global} \in \mathbb{R}^{132}$ is computed by multiplying local rotation matrices of joints following the hierarchy of the structure of the SMPL skeleton (Pavlakos et al., 2019). The two pose vectors are then concatenated and fed into the LSTM initialization model. We utilize five Model Initialization modules to generate the initial hidden states $h_0^B = \{h_0^B\}_{i=1}^5 \in \mathbb{R}^{5 \times h}$ and cell states $c_0^B = \{c_0^B\}_{i=1}^5 \in \mathbb{R}^{5 \times h}$ for all five LSTMs in the Spatial-Temporal Backbone, where h represents the hidden size of the LSTM layer. Each Model Initialization module is implemented as an MLP, which consists of three fully connected layers, with ReLU activation functions applied between the layers. The dimension of the MLP's intermediate hidden layer is 512.

Body-part motion embedding. Following previous methods (Aliakbarian et al., 2023; Dai et al., 2024; Jiang et al., 2022a), we process the sparse positions and rotations to obtain the position, rotation, linear velocity, and angular velocity for each body part and for each hand relative to the head coordinate frame. For each frame, we use the following representations for the head, hands, and hands in the head coordinate frame: (1) positions: $\{\mathbf{p}_h, \mathbf{p}_l, \mathbf{p}_r, \mathbf{p}_{lh}, \mathbf{p}_{rh}\}$; (2) rotations: $\{\mathbf{r}_h, \mathbf{r}_l, \mathbf{r}_r, \mathbf{r}_{lh}, \mathbf{r}_{rh}\}$; (3) linear velocities $\{\mathbf{v}_h, \mathbf{v}_l, \mathbf{v}_r, \mathbf{v}_{lh}, \mathbf{v}_{rh}\}$; (4) angular velocities $\{\omega_h, \omega_l, \omega_r, \omega_{lh}, \omega_{rh}\}$. The feature representations for each body part are organized as follows: $\mathbf{f}_h = \{\mathbf{p}_h, \mathbf{r}_h, \mathbf{v}_h, \omega_h\}$, $\mathbf{f}_l = \{\mathbf{p}_l, \mathbf{r}_l, \mathbf{v}_l, \omega_l\}$, $\mathbf{f}_r = \{\mathbf{p}_r, \mathbf{r}_r, \mathbf{v}_r, \omega_r\}$, $\mathbf{f}_{lh} = \{\mathbf{p}_{lh}, \mathbf{r}_{lh}, \mathbf{v}_{lh}, \omega_{lh}\}$, $\mathbf{f}_{rh} = \{\mathbf{p}_{rh}, \mathbf{r}_{rh}, \mathbf{v}_{rh}, \omega_{rh}\}$. We then employ five embedding modules to represent the features of each body part: $e_h, e_l, e_r, e_{lh}, e_{rh}$, which are fed into the Spatial-Temporal Backbone for feature extraction. Each embedding module consists of four 128-dimensional fully connected layers, which encode the pose, rotational velocity, position, and linear velocity of each body part, respectively. Each fully connected layer is followed by a LeakyReLU activation function.

Spatial-temporal backbone. In the Spatial-Temporal Backbone, the embedding sequences of each body part are first individually processed

through LSTMs to extract temporal features. The temporal feature sequences from each LSTM are then treated as individual tokens, which are concatenated into a sequence and fed into the transformer encoder. The transformer encoder enables inter-token interaction and models the spatial dependencies between different body parts. The Spatial-Temporal Backbone can be stacked into multiple layers to extract deep Spatial-Temporal features from sparse inputs. In the Spatial-Temporal Backbone, each LSTM is a single-layer unidirectional architecture with a hidden size of 512. The transformer encoder in the Backbone has three stacked layers. Each layer has eight attention heads and a feed-forward layer, with a hidden size of 512.

Temporal regression. The features from the Spatial-Temporal Backbone are fed into two parallel modules: the Body Pose Header and the SMPL Shape Header, which output sequences of full-body local poses $\hat{\Theta}$ and SMPL shape parameters $\hat{\beta}$ (Loper et al., 2023), respectively. Since the body-part motion embedding sequences in the backbone are processed as individual tokens by the transformer encoder through self-attention mechanisms, the inherent temporal coherence of the sequences may be compromised. To address this problem, we concatenate the output sequences of all body parts $\{e'_h, e'_l, e'_r, e'_{lh}, e'_{rh}\}$ frame-by-frame, forming a unified feature sequence f^B , which is then processed by an LSTM for temporal modeling. This LSTM module is shared by both headers. Each header employs its own MLPs to output pose and shape sequences, respectively. The LSTM module is a single-layer unidirectional architecture with a hidden size of 2560. The MLP in each regression header consists of two fully connected layers, with LeakyReLU activation functions applied between the layers, and a hidden layer dimension of 256. The output feature dimensions of the two MLPs are 132 and 16, respectively.

Loss functions. The output and ground-truth pose sequences are denoted as $\hat{\Theta}$ and Θ , respectively. In the training stage of RIM, Forward-Kinematics is applied to both pose sequences to compute the global joint rotation sequence $\hat{\Theta}_{glb}$ and the position sequence $\hat{X}$, as well as the ground-truth Θ_{glb} and X . The root joint rotation sequences $\hat{r}$ and r are obtained from $\hat{\Theta}_{glb}$ and Θ_{glb} . The joint acceleration sequences $\hat{a}$ and a are calculated from $\hat{X}$ and X . The terms of the loss function include local poses constraint $\mathcal{L}_{local}$, global poses constraint $\mathcal{L}_{glb}$, joint positions constraint $\mathcal{L}_{pos}$, root joint orientation constraint $\mathcal{L}_{root}$, acceleration constraint $\mathcal{L}_{acc}$, and SMPL (Loper et al., 2023) shape constraint $\mathcal{L}_{shape}$:

$$\mathcal{L}_{local} = \|\hat{\Theta} - \Theta\|_1, \quad (5)$$

$$\mathcal{L}_{glb} = \|\hat{\Theta}_{glb} - \Theta_{glb}\|_1, \quad (6)$$

$$\mathcal{L}_{pos} = \|\hat{X} - X\|_1, \quad (7)$$

$$\mathcal{L}_{root} = \|\hat{r} - r\|_1, \quad (8)$$

$$\mathcal{L}_{acc} = \|\hat{a} - a\|_1, \quad (9)$$

$$\mathcal{L}_{shape} = \|\hat{\beta} - \beta\|_1. \quad (10)$$

The loss function for training RIM is defined as follows, which is consistent with HMD-Poser (Dai et al., 2024):

$$\mathcal{L} = \alpha_{local}\mathcal{L}_{local} + \alpha_{glb}\mathcal{L}_{glb} + \alpha_{pos}\mathcal{L}_{pos} + \alpha_{root}\mathcal{L}_{root} + \alpha_{acc}\mathcal{L}_{acc} + \alpha_{shape}\mathcal{L}_{shape}, \quad (11)$$

where α_{local} , α_{glb} , α_{pos} , α_{root} , α_{acc} are hyper-parameters.

3.4. Recurrent inference process

In the inference stage of RIM, we account for the temporal dependencies between subsequences. A motion sequence of length L is divided into N subsequences with a length of n ($n = 40$). When inferring the first subsequence, the LSTM initialization described in Section 3.3 is executed first. After the inference of the first sub-sequence, the hidden states h_0^B , h_0^H and cell states c_0^B , c_0^H of all LSTMs are preserved and used as the initial states for the inference with the next sub-sequence. Through this approach, the LSTMs effectively perform continuous inference over the entire sequence, significantly improving temporal smooth-

ness in the generated motion compared to independent subsequence-wise inference (see Section 4.5). The inference process of RIM is described in Algorithm 2, where the Model Initialization, Body-Part Motion Embedding, Spatial-Temporal Backbone, and Temporal Regression modules are named *Init*, *EMB*, *Backbone*, *Temp*, respectively.

Algorithm 2 Inference process of RIM.

Require: input motion sequence $S = \{S_i\}_{i=1}^N$; full-body initial poses Θ_0 ; model Initialization module *Init*; body-part motion embedding module *EMB*; spatial-temporal backbone module *Backbone*; temporal regression module *Temp*;

- 1: **for** each $i \in [1, N]$ **do**
- 2: **if** $i = 1$ **then**
- 3: Run Model Initialization: $h_0^B, c_0^B = \text{Init}(\Theta_0, FK(\Theta_0))$
- 4: Set initial states h_0^H and c_0^H of the LSTM model in Temporal Regression as None
- 5: **end if**
- 6: Run Body-Part Motion Embedding: $e_h, e_l, e_r, e_{lh}, e_{rh} = \text{EMB}(S_i)$
- 7: Run Spatial-Temporal Backbone: $f_i^B, h_0^B, c_0^B = \text{Backbone}(e_h, e_l, e_r, e_{lh}, e_{rh}, h_0^B, c_0^B)$
- 8: Run Temporal Regression: $\Theta_i, \beta_i, h_0^H, c_0^H = \text{Temp}(f_i^B, h_0^H, c_0^H)$
- 9: **end for**

Output: $\Theta = \{\Theta_1, \dots, \Theta_N\}$, $\beta = \{\beta_1, \dots, \beta_N\}$

4. Experiments

4.1. Datasets and metrics

Datasets. In our experiments, we utilized two categories of datasets: (1) the AMASS dataset (Mahmood et al., 2019), which provides synthesized HMD and controller data inferred from SMPL motion sequences (Loper et al., 2023); (2) the FreeDancing (Dai et al., 2024) and EMHI (Fan et al., 2025) datasets, which contain real HMD and controller data with paired full-body human motion sequences. All datasets were resampled at a frame rate of 60 Hz. Specifically, the AMASS dataset (Mahmood et al., 2019) includes three subsets: CMU (Cmu, 2025), BMLr (Troje, 2001) and HDM05 (Müller et al., 2007). Following prior work (Barquero et al., 2025; Dai et al., 2024; Du et al., 2023; Jiang et al., 2022a), we trained RIM using the training set of *Protocol 1* (approximately 3M frames). Additionally, the FreeDancing dataset (approximately 0.55M frames) and the EMHI dataset (approximately 3M frames) served as the testing set for evaluating the motion generation model. When comparing different motion data augmentation strategies (see Section 4.4), we randomly selected 50% (approximately 0.28M frames) of the FreeDancing dataset as the training set and used EMHI as the testing set. This dataset split results in a training set with a size of less than 10% of the EMHI testing dataset to demonstrate the generalization capabilities of different strategies. Using different datasets as the training and testing sets mitigates the overfitting risks that may arise if fine-tuning and testing are performed on the same dataset, as the motion data in a dataset are collected using identical hardware under the same experimental conditions and may exhibit fixed noise patterns. FreeDancing and EMHI are collected from different subjects, with different hardware setups and capture environments. This ensures a rigorous evaluation of the generalization capability of trained models.

Metrics. In line with previous work (Dai et al., 2024; Du et al., 2023), we employ the following eight metrics to evaluate motion generation: (1) MPJRE (Mean Per Joint Rotation Error [°]); (2) MPJPE (Mean Per Joint Position Error [cm]); (3) MPJVE (Mean Per Joint Velocity Error [cm/s]); (4) Jitter [$10^2 m/s^3$], which measures the average rate of change of the acceleration of each joint; (5) H-PE (Hands Position Error [cm]); (6) U-PE (Upper Body Position Error [cm]); (7) L-PE (Lower Body Position Error [cm]); (8) R-PE (Root Position Error [cm]);

Table 1

Comparative results on the FreeDancing dataset. The best values are marked in bold. Second-best values are underlined. ↓ indicates that a lower value of the metric corresponds to better model performance.

Method	MPJRE↓	MPJPE↓	MPJVE↓	Jitter↓	H-PE↓	U-PE↓	L-PE↓	R-PE↓
AvatarPoser	7.18	9.78	43.30	24.31	9.39	5.67	15.72	8.01
AGRoL	7.89	12.56	54.17	45.81	11.54	8.50	18.42	10.80
HMD-Poser	7.25	10.02	38.42	15.23	9.20	6.14	15.63	9.10
RPM	8.53	11.88	33.12	9.77	17.55	8.55	16.70	8.69
RIM (Ours)	7.13	9.56	<u>33.19</u>	<u>11.36</u>	9.34	<u>5.82</u>	14.96	7.86

Table 2

Comparative results on the EMHI dataset. The best values are marked in bold. Second-best values are underlined. ↓ indicates that a lower value of the metric corresponds to better model performance.

Method	MPJRE↓	MPJPE↓	MPJVE↓	Jitter↓	H-PE↓	U-PE↓	L-PE↓	R-PE↓
AvatarPoser	8.10	8.59	42.61	19.64	6.83	4.61	14.35	6.81
AGRoL	8.76	13.01	53.70	40.09	14.03	9.77	17.69	11.20
HMD-Poser	8.19	8.84	38.99	12.92	9.36	5.35	13.89	6.89
RPM	8.15	12.12	38.81	10.54	18.68	9.10	16.47	10.20
RIM (Ours)	7.95	8.27	33.95	9.49	<u>8.62</u>	<u>4.79</u>	13.31	6.46

4.2. Implementation details

The subsequence length n was set as 40 for both the noise generation model and RIM, in both the training and evaluation stages. The proposed approach was implemented using PyTorch 1.11.0 and all experiments were run on an AutoDL deep learning server equipped with an AMD EPYC 9754 CPU and an NVIDIA RTX 4090 GPU with 24 GB graphics memory. We employed the Adam (Kingma & Ba, 2014) optimizer with a batch size of 256 for all datasets and trained the noise generation model for 20k iterations (approximately 80 epochs) and the RIM model for 20 epochs. We set α_{kl} , α_{local} , α_{glb} , α_{pos} , α_{root} , α_{acc} , α_{shape} to 0.001, 5.0, 1.0, 1.0, 1.0, 0.5, and 0.1, respectively.

The Positional and Rotational Noise Encoder $\mathbb{E}_n$, Conditional Positional and Rotational Encoder $\mathbb{E}_c$, Positional Noise Decoder $\mathbb{D}_p$, and Rotational Noise Decoder $\mathbb{D}_r$, are all implemented as three-layer MLPs with a 512-dimensional hidden layer. All layers in each MLP are preceded by a dropout layer ($p=0.25$) and the first two layers are followed by ELU activation functions. The μ Head $\mathbb{H}_\mu$ and σ Head $\mathbb{H}_\sigma$ are both implemented as two-layer MLPs with the hidden size of 512. The LeakyReLU activation function is applied after the first fully connected layer. μ , σ , and z are all 512-dimensional vectors. The noise feature $\mathbf{f}_n$ and condition feature $\mathbf{f}_c$ are both 512-dimensional ($D = F = 512$).

4.3. Comparisons with HMD-based motion generation methods

We compared the proposed RIM with HMD-based methods (Barquero et al., 2025; Dai et al., 2024; Du et al., 2023; Jiang et al., 2022a) on FreeDancing and EMHI datasets. All models were trained on the AMASS dataset (Mahmood et al., 2019) without the data augmentation strategy. As shown in Table 1, RIM surpasses all baseline methods in terms of MPJRE and MPJPE, while achieving comparable performance to RPM in MPJVE. As shown in Table 2, RIM achieves the lowest values in MPJRE, MPJPE, and MPJVE, while also attaining the lowest Jitter. Experimental results demonstrate that, without using specific training strategies, our method achieves the best overall performance on FreeDancing and EMHI datasets with real device data. Fig. 4 visualizes the qualitative results of our method compared to previous baseline methods, which shows that our method can generate accurate and smooth full-body pose sequences.

We further compared the HMD-based methods in terms of computational costs, model parameters, model sizes, and inference speeds, which are important for real-time applications. All methods were run on an

Table 3

Comparisons with the HMD-based methods in terms of computational costs, model parameters, model sizes, and inference speeds. The best values are marked in bold. ↓ indicates that a lower value of the metric corresponds to better model performance. ↑ indicates that a higher value corresponds to better performance.

Method	FLOPs (G)↓	Params (M)↓	Model Size (MB)↓	FPS↑
AvatarPoser	0.63	3.34	15.75	204.16
AGRoL	0.84	7.04	36.66	102.77
HMD-Poser	2.40	15.87	66.63	111.21
RPM	0.35	4.37	47.51	877.22
RIM (Ours)	5.53	89.32	375.44	81.30

NVIDIA GeForce RTX 4090D device in this experiment. As shown in Table 3, our method satisfies the requirement of real-time applications.

4.4. Evaluations on motion data augmentation strategies

We compared the performance of the following four training strategies: (a) Direct: the motion generation model was trained directly on the synthetic motion dataset and tested on real device data. (b) Fine-tuning: the motion generation model was pre-trained on the synthetic motion dataset and was fine-tuned and tested on real device data. (c) Additive: all conditional subsequences $\{\mathbf{P}_{\text{body}}, \mathbf{R}_{\text{body}}\}$ in the real device training set were initialized using the Faiss (Douze et al., 2026) library by indexing before training the downstream motion generation model; during the training phase, conditional subsequences in the synthetic motion dataset were used to retrieve their most similar records via index searching from the Faiss index, thereby obtaining the corresponding positional and rotational noises from the real device training set that best matched the positions and rotations from the synthetic motion dataset; the sampled noises were applied to each batch of synthetic motion data; the trained model was tested on real device data. (d) Generative (the proposed approach): each batch of synthetic motion data was augmented using the positional and rotational noises generated from the pre-trained motion noise generation model during the training stage; the trained model was tested on real device data. We randomly sampled 50% of the FreeDancing dataset (approximately 0.28M frames) as the training set for fine-tuning, additive and generative strategy. All strategies are illustrated in Fig. 5.

As shown in Table 4, the proposed generative strategy yields the best overall performance for methods, including AGRoL, HMD-Poser, RPM and RIM. Notably, RIM achieves the best results for nearly all metrics

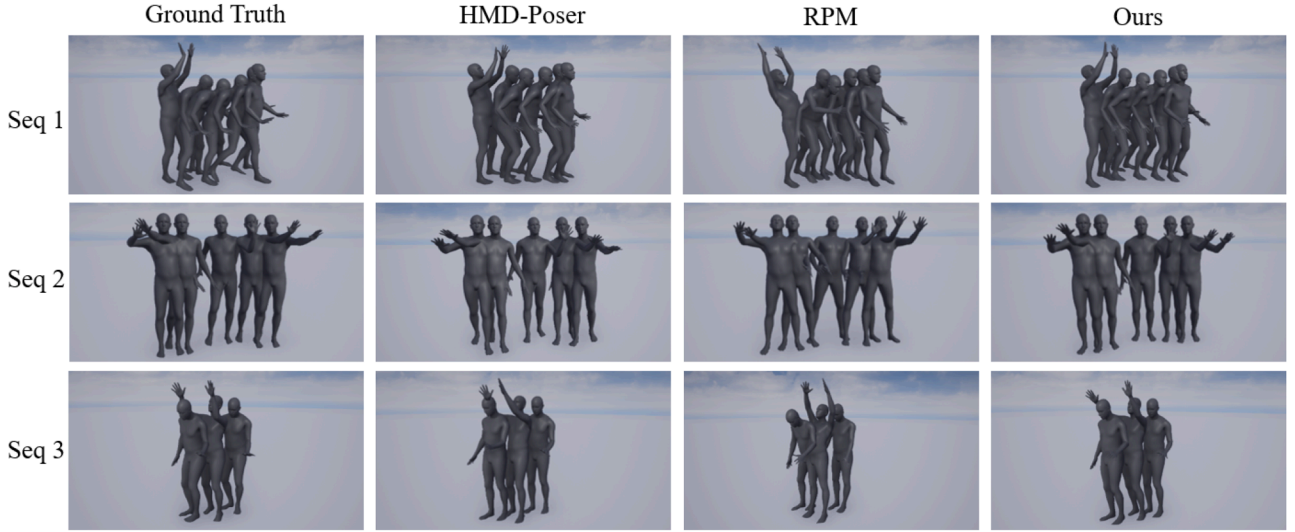


Fig. 4. Qualitative comparisons between our method and HMD-based SOTA methods on the EMHI dataset.

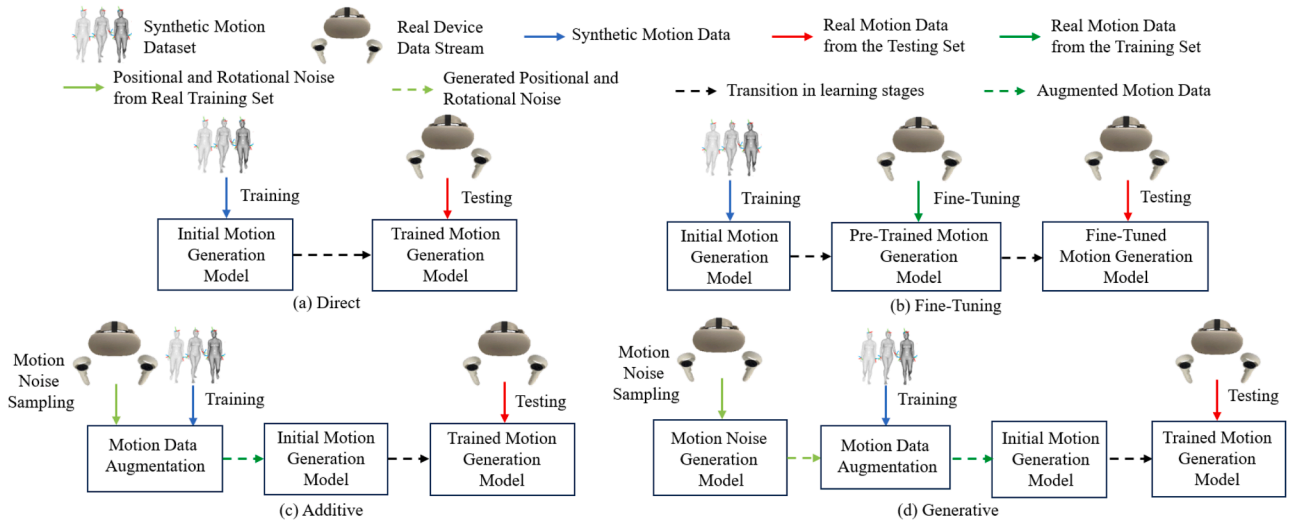


Fig. 5. Illustration of four different training strategies: (a) Direct: the motion generation model was trained directly on the synthetic motion dataset and tested on real device data. (b) Fine-tuning: the motion generation model was pre-trained on the synthetic motion dataset and was fine-tuned and tested on real device data. (c) Additive: positional and rotational noises were sampled from real device data and applied to each batch of synthetic motion data during the training stage; the trained model was tested on real device data. (d) Generative: each batch of synthetic motion data was augmented using the positional and rotational noises generated from the pre-trained motion noise generation model during the training stage; the trained model was tested on real device data.

when employing the generative strategy and consistently outperforms baseline motion generation methods with their given strategies.

Compared to other strategies, fine-tuning may lead to overfitting on a single dataset (FreeDancing), thereby compromising generalization capability and resulting in performance inferior to the direct strategy on most metrics. The additive strategy, which samples noise from real-captured datasets, improves model performance over the direct strategy. However, it can only sample noise with limited patterns, leading to insufficient noise diversity. In contrast, the generative strategy can sample more diverse noise from the latent space, which enables the downstream motion generation model to achieve better performance.

4.5. Ablation studies

To evaluate the effectiveness of key components in RIM, we conducted ablation studies on (1) effect of *Model Initialization*, which is evaluated by removing the *Model Initialization* module (w/o Init), (2)

effect of *Temporal Modeling* in Temporal Regression, which is evaluated by removing the LSTM module (w/o Temp), and (3) effect of *Recurrent Inference*, which is evaluated by removing the inter-subsequence transitions of hidden states and cell states of LSTMs in either the Backbone or Temporal Regression between sub-sequences (w/o B States, w/o T States).

As shown in Table 5, both *Model Initialization* and *Temporal Modeling* improved overall performance of motion generation in terms of lower errors and Jitter. These improvements remain consistent across both the direct strategy and the generative strategy. Moreover, the state transition mechanism of LSTMs in the Backbone effectively enhances the model's performance in terms of temporal smoothness. The LSTM in Temporal Regression plays an important role in maintaining inter-subsequence inference consistency. When the LSTM in Temporal Regression performs independent inference for each subsequence without initialization using the previous states, the model's performance on temporal smoothness degrades.

Table 4

Comparative results of different strategies tested on the EMHI dataset. † marks the best values of training strategies for each motion generation method. The best values over all motion generation methods and training strategies are marked in bold. ↓ indicates that a lower value of the metric corresponds to better model performance.

Method	Strategy	MPJRE↓	MPJPE↓	MPJVE↓	Jitter↓	H-PE↓	U-PE↓	L-PE↓	R-PE↓
AvatarPoser	Direct	8.10†	8.59	42.61	19.64	6.83†	4.61†	14.35	6.81
	Fine-tuning	8.26	8.99	41.85	19.03	7.68	4.90	14.90	6.55
	Additive	8.26	8.55	40.12†	17.33†	7.41	5.00	13.69†	6.52
	Generative	8.50	8.51†	41.01	18.27	7.21	4.73	13.97	6.39†
AGRoL	Direct	8.76	13.01	53.70	40.09	14.03	9.77	17.69	11.20
	Fine-tuning	9.48	14.92	50.42	35.96	16.40	10.53	21.26	11.20
	Additive	8.72†	12.62	49.60	36.42	13.40	9.30	17.43	10.61
	Generative	8.86	11.84†	48.31†	34.03†	11.39†	8.25†	17.03†	9.68†
HMD-Poser	Direct	8.19	8.84	38.99	12.92†	9.36	5.35	13.89	6.89
	Fine-tuning	8.00†	9.91	37.04†	13.92	13.24	6.87	14.31	7.21
	Additive	8.11	8.40	37.88	13.84	8.16	4.98	13.35	6.68
	Generative	8.09	8.17†	38.73	13.53	8.10†	4.84†	12.98†	6.54†
RPM	Direct	8.15	12.12	38.81	10.54	18.68	9.10	16.47	10.20
	Fine-tuning	9.16	15.16	42.94	9.78†	24.16	11.22	20.86	11.93
	Additive	7.81†	10.72	36.30†	10.63	15.11†	7.65	15.17	9.04
	Generative	7.91	10.61†	38.25	10.54	15.50	7.64†	14.90†	8.38†
RIM (Ours)	Direct	7.95	8.27	33.95	9.49	8.62	4.79	13.31	6.46
	Fine-tuning	7.94	8.66	33.31	11.25	9.60	5.04	13.88	6.58
	Additive	7.87	8.76	32.95	9.16	9.44	5.44	13.57	7.02
	Generative	7.72†	7.69†	32.44†	9.02†	7.79†	4.53†	12.26†	5.78†

Table 5

Results of the ablation study on *Model Initialization*, *Temporal Regression* and *Recurrent Inference*. The best values are marked in bold. Second-best values are underlined. ‡ denotes ties. ↓ indicates that a lower value of the metric corresponds to better model performance.

Protocol	Method	MPJRE↓	MPJPE↓	MPJVE↓	Jitter↓
FreeDancing with Direct Strategy	w/o Init	7.44	10.17	39.11	12.19
	w/o Temp	7.25	10.35	35.11	<u>12.09</u>
	w/o B States	7.11	9.47	<u>33.81</u>	12.76
	w/o T States	7.16	11.79	226.52	215.05
	Original RIM	<u>7.13</u>	<u>9.56</u>	33.19	11.36
EMHI with Direct Strategy	w/o Init	8.22	8.77	38.40	10.43
	w/o Temp	8.13	8.27‡	34.69	9.04
	w/o B States	7.95‡	8.28	34.94	11.07
	w/o T States	8.04	10.80	240.11	227.67
	Original RIM	7.95‡	8.27‡	33.95	<u>9.49</u>
EMHI with Generative Strategy	w/o Init	8.18	8.55	37.13	10.03
	w/o Temp	8.04	<u>8.10</u>	<u>34.39</u>	<u>9.78</u>
	w/o B States	<u>7.77</u>	8.41	34.69	12.23
	w/o T States	8.14	10.44	227.44	221.58
	Original RIM	7.72	7.69	32.44	9.02

5. Conclusion and future work

In this paper, we proposed the generative motion data augmentation strategy and the RIM model to reliably generate full-body pose sequences and SMPL shape parameters (Loper et al., 2023) based on the tracking data from an HMD and its two controllers. We demonstrated the effectiveness of both the generative motion data augmentation strategy and the RIM model through experiments and further confirmed the validity of the key components of RIM via ablation studies. The generative motion data augmentation strategy is capable of effectively learning noise patterns from a relatively small-scale noise dataset and generalizing to a large-scale dataset. The performance of multiple downstream motion generation models were improved. The proposed RIM architecture achieves superior performance in generating accurate and smooth human motions compared to existing methods.

A limitation of our generative motion data augmentation strategy is that the noise generation is performed on every batch of training data during training. This inevitably introduces computational and resource overhead during the training stage of downstream motion generation model. Future research could be conducted from several different aspects. For example, it would be feasible to design new noise generation models based on other generative methods, such as diffusion (Ho

et al., 2020) or normalizing flows (Rezende & Mohamed, 2015), and compare their performance with the current VAE-based approach. For downstream human motion generation models, generative methods and reinforcement learning can be further explored. In addition, IMUs are also often not rigidly attached to human body when generating human motion based on IMUs. It would be sensible to apply the approach proposed in this paper to IMU-based human motion generation and test whether the proposed approach would improve the performance in such cases.

CRediT authorship contribution statement

Zihao Guo: Data curation, Investigation, Methodology, Software, Writing – original draft; **Jingbo Zhao:** Conceptualization, Methodology, Project administration, Resources, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Supplementary material

Supplementary material associated with this article can be found in the online version at [10.1016/j.neunet.2026.108918](https://doi.org/10.1016/j.neunet.2026.108918).

References

- Aliakbarian, S., Cameron, P., Bogo, F., Fitzgibbon, A., & Cashman, T. J. (2022). FLAG: Flow-based 3D avatar generation from sparse observations. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 13253–13262).
- Aliakbarian, S., Saleh, F., Collier, D., Cameron, P., & Cosker, D. (2023). HMD-NeMo: Online 3D avatar motion generation from sparse observations. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 9622–9631).
- Barquero, G., Bertsch, N., Marramreddy, M., Chacón, C., Arcadu, F., Rigual, F., He, N. S., Palmero, C., Escalera, S., Ye, Y. et al. (2025). From sparse signal to smooth motion: Real-time motion generation with rolling prediction models. In *Proceedings of the computer vision and pattern recognition conference* (pp. 1850–1860).
- Cmu, Mocap dataset, <http://mocap.cs.cmu.edu/>, last accessed May 2025.
- Dai, P., Zhang, Y., Liu, T., Fan, Z., Du, T., Su, Z., Zheng, X., & Li, Z. (2024). HMD-Poser: On-device real-time human motion tracking from scalable sparse observations. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 874–884).
- Dittadi, A., Dziadzio, S., Cosker, D., Lundell, B., Cashman, T. J., & Shotton, J. (2021). Full-body motion from a single head-mounted device: Generating smpl poses from partial observations. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 11687–11697).
- Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.-E., Lomeli, M., Hosseini, L., & Jégou, H. (2026). The faiss library. *IEEE Transactions on Big Data*, 12(2), 346–361.
- Du, Y., Kips, R., Pumarola, A., Starke, S., Thabet, A., & Sanakoyeu, A. (2023). Avatars grow legs: Generating smooth human motion from sparse tracking inputs with diffusion model. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 481–490).
- Fan, Z., Dai, P., Su, Z., Gao, X., Lv, Z., Zhang, J., Du, T., Wang, G., & Zhang, Y. (2025). EMH: A multimodal egocentric human motion dataset with HMD and body-worn IMUs. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 2879–2887). (vol. 39).
- Ho, J., Jain, A., & Abbeel, P. (2020). Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33, 6840–6851.
- Jiang, J., Strelci, P., Qiu, H., Fender, A., Laich, L., Snape, P., & Holz, C. (2022a). AvatarPoser: Articulated full-body pose tracking from sparse motion sensing. In *European conference on computer vision* (pp. 443–460). Springer.
- Jiang, Y., Ye, Y., Gopinath, D., Won, J., Winkler, A. W., & Liu, C. K. (2022b). Transformer inertial poser: Real-time human motion reconstruction from sparse IMUs with simultaneous terrain generation. In *Siggraph asia 2022 conference papers* (pp. 1–9).
- Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv:1412.6980.
- Kullback, S., & Leibler, R. A. (1951). On information and sufficiency. *Ann. Math. Stat.*, 22(1), 79–86.
- Lee, S., Starke, S., Ye, Y., Won, J., & Winkler, A. (2023). QuestEnvSim: Environment-aware simulated motion tracking from sparse sensors. In *ACM siggraph 2023 conference proceedings* (pp. 1–9).
- Li, Z., Yu, C., Liang, C., & Shi, Y. (2024). Poseaugment: Generative human pose data augmentation with physical plausibility for IMU-based motion capture. In *European conference on computer vision* (pp. 55–73). Springer.
- Loper, M., Mahmood, N., Romero, J., Pons-Moll, G., & Black, M. J. (2023). SMPL: A skinned multi-person linear model. In *Seminal graphics papers: Pushing the boundaries, volume 2* (pp. 851–866).
- Mahmood, N., Ghorbani, N., Troje, N. F., Pons-Moll, G., & Black, M. J. (2019). AMASS: Archive of motion capture as surface shapes. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 5442–5451).
- Müller, M., Röder, T., Clausen, M., Eberhardt, B., Krüger, B., & Weber, A. (2007). Documentation mocap database HDM05. Technical Report No. CG-2007-2, Institut für Informatik II, Universität Bonn. https://resources.mpi-inf.mpg.de/HDM05/07_MuRoCIEbKrWe_HDM05.txt
- Patel, C., Liao, Z., & Pons-Moll, G. (2020). TailorNet: Predicting clothing in 3d as a function of human pose, shape and garment style. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 7365–7375).
- Pavlakos, G., Choutas, V., Ghorbani, N., Bolkart, T., Osman, A. A. A., Tzionas, D., & Black, M. J. (2019). Expressive body capture: 3D Hands, face, and body from a single image. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 10975–10985).
- Rezende, D., & Mohamed, S. (2015). Variational inference with normalizing flows. In *International conference on machine learning*, vol. 37 (pp. 1530–1538).
- Sohn, K., Yan, X., & Lee, H. (2015). Learning structured output representation using deep conditional generative models. In *Proceedings of the 29th international conference on neural information processing systems*, Vol. 2 (pp. 3483–3491).
- Starke, S., Starke, P., He, N., Komura, T., & Ye, Y. (2024). Categorical codebook matching for embodied character controllers. *ACM Transactions on Graphics (TOG)*, 43(4), 1–14.
- Troje, N. F. (2001). Decomposing biological motion: A linear model for analysis and synthesis of human gait patterns. *Journal of Vision*, 1(3), 355.
- Van Wouwe, T., Lee, S., Falisse, A., Delp, S., & Liu, C. K. (2024). DiffusionPoser: Real-time human motion reconstruction from arbitrary sparse sensors using autoregressive diffusion. In *2024 IEEE/CVF Conference on computer vision and pattern recognition* (pp. 2513–2523).
- Winkler, A., Won, J., & Ye, Y. (2022). QuestSim: Human motion tracking from sparse sensors with simulated avatars. In *Siggraph Asia 2022 conference papers* (pp. 1–8).
- Yi, X., Zhou, Y., Habermann, M., Shimada, S., Golyanik, V., Theobalt, C., & Xu, F. (2022). Physical inertial poser (pip): Physics-aware real-time human motion tracking from sparse inertial sensors. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 13167–13178).
- Zhang, Y., Xia, S., Chu, L., Yang, J., Wu, Q., & Pei, L. (2024). Dynamic inertial poser (dynaIP): Part-based motion dynamics learning for enhanced human pose estimation with sparse inertial sensors. In *2024 IEEE/CVF Conference on computer vision and pattern recognition* (pp. 1889–1899).
- Zheng, X., Su, Z., Wen, C., Xue, Z., & Jin, X. (2023). Realistic full-body tracking from sparse observations via joint-level modeling. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 14678–14688).
- Zhou, Y., Barnes, C., Lu, J., Yang, J., & Li, H. (2019). On the continuity of rotation representations in neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 5745–5753).
- Zuo, C., Wang, Y., Zhan, L., Guo, S., Yi, X., Xu, F., & Qin, Y. (2024). Loose inertial poser: Motion capture with IMU-attached loose-wear jacket. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 2209–2219).